FOX MLO // TRANSLATION WORKFLOW ORCHESTRATOR // PLUNET + POLYLANG

UX Redesign & Implementation Plan

This document records the agreed UX direction for FOX MLO: the domain model, the screens, the workflow state model and controls, the Polylang mock server, and the implementation phases. The visual theme (Swiss style, Plunet cyan, square geometry, mono data) stays as it is. Only structure and flows change.

1. Goal

FOX MLO is a workflow orchestrator. A user creates an **Auftrag** (order) or an **Angebot** (quote) in Plunet from content held in WordPress/Polylang. The Auftrag has one or more **Positions**; each Position carries one .xliff item and runs one **Workflow** made of asynchronous **Tasks** (PM, translation, review, QA, ...). The user must be able to:

- **Observe** the state of every Workflow and Task.
- **Monitor** what happens through live logs.
- **Assess** the work through analytics: duration, cost, errors, per order, per PM, per customer.
- **Control** execution: pause, resume, cancel, retry, restart from a task.

Main journey: **create an Auftrag → open it → browse its Positions → follow each Workflow.**

2. What is wrong with the current UX

PROBLEM	EVIDENCE IN THE CODE TODAY
No domain hierarchy	Everything is one flat `ETLJob` = one XLIFF. There is no Auftrag/Angebot, Position, Project Manager or customer.
Home page is a launcher	A marketing hero plus a “Dispatch pipeline” form. Work in progress sits in a tab below.
Fire-and-forget execution	`POST /etl/run` runs Extract → Validate → Plunet → Deliver in one HTTP request. There is nothing to observe or control.
No control, logs or analytics	No pause/cancel/restart. The only log is `error_message`.
Engineer-oriented detail page	Segments and raw/processed XML are the main content, not “where is this and what does it need”.
Opaque choices	“Workflow Type 1/2/3” and “Wordpress Server 1/2” carry no meaning. Infrastructure badges dominate the header.
Monolithic frontend	`App.tsx` is one 1,045-line component with view state instead of routes.

3. Domain model & terminology

Plunet already uses the word *Job* for a single step on an Item. To avoid a clash, the UI uses the following names (agreed):

```
Auftrag (Plunet Order)            one Project Manager, one customer, deadline
 ■■ Position (Plunet Item)           1 post × 1 target language = 1 .xliff
     ■■ Workflow                       one per Position, built from a template
         ■■ Task (Plunet Job)          PM · TRA · REV · QA · ... + Import / Deliver

Angebot (Plunet Quote)            same shape, but Workflows do NOT run
  ■■ accepted → becomes an Auftrag → Workflows start
```

CONCEPT	RULE
Position	Exactly one post, one target language, one single .xliff file.
Project Manager	Chosen from **Plunet resources**. One per Auftrag.
Angebot	Has Positions and a Workflow preview, but never runs Tasks. Acceptance can happen **in the app** (Accept button, which converts the quote to an order in Plunet) **or in Plunet** (the app detects it via Plunet callbacks or polling and converts it locally).
Workflow template	Chosen per Auftrag (e.g. *Standard: Import → PM → TRA → REV → QA → Deliver*). It can be overridden per Position: add, remove or reorder Tasks.
Task types	PM, Translation, Review, QA, extensible with other Plunet job types. Import (from Polylang) and Deliver (return trip) are orchestrator Tasks.

4. Decisions log

#	QUESTION	DECISION
1	Where does XLIFF come from?	From the Polylang API. The mock serves ready-made .xliff files.
2	Return trip?	Flexible. Default: push back into WordPress as a linked translated post (`translations[...]`). Options: **Polylang (default)**, **other endpoint**, **none (download only)**. Set per Auftrag, overridable per Position.
3	What is a Position?	One post × one target language = one single XLIFF.
4	Async tasks	PM, Translation, Review, QA, plus more from the Plunet API doc.
5	Angebot	Runs no Workflow. Becomes an Auftrag once accepted, from the app or from Plunet.
6	Control semantics	Industry-standard semantics; see section 6.
7	Project Manager	From Plunet resources. No login yet, but the design must allow auth/roles later.
8	Analytics	Across orders: duration, cost, errors; per PM and per customer.
9	Naming	Auftrag → Position → Workflow → Tasks.
10	UI language	Bilingual: **English and French**, with a language switch.

5. Information architecture & screens

```
Header:  FOX MLO  ·  Aufträge  ·  Analytics  ·  [health dot]  ·  [EN | FR]

/orders                          Aufträge list (home)
/orders/new                      New Auftrag / Angebot wizard
/orders/:orderId                 Auftrag detail (Positions)
/orders/:orderId/positions/:pid  Position / Workflow detail
/analytics                       Cross-order analytics
```

5.1 Aufträge list (home)

- Table: Plunet number, type (Auftrag/Angebot), customer, Project Manager, Position count, combined status bar (e.g. 3/4 done, 1 failed), deadline, last activity.
- Filters: type, status, PM, customer, and a **Needs attention** toggle (failed, paused, overdue).
- Primary action: **+ New Auftrag**. The marketing hero is removed; system health becomes a small dot in the header.

5.2 New Auftrag wizard

- **Step 1 – Basics:** Auftrag or Angebot, customer, Project Manager (Plunet resources), deadline.
- **Step 2 – Source:** WordPress/Polylang site → source language → select posts → select target languages. Every post × target language pair becomes one Position (live count shown). Manual .xliff upload stays as a fallback.
- **Step 3 – Workflow:** template for the Auftrag, with the Task chain previewed. Per-Position override. Return-trip destination (Polylang / other endpoint / none).
- **Step 4 – Review:** summary (Positions, words, estimated cost). **Create** registers the order or quote in Plunet; for an Auftrag the Workflows start.

5.3 Auftrag detail

- Header: Plunet number, type, customer, PM, deadline, overall progress, Plunet link. For an Angebot: **Accept → convert to Auftrag** and Reject.
- Positions table: post title, language pair, words, Workflow status, inline Task stepper (Import ✓ → PM ✓ → TRA ● → REV – → QA – → Deliver –; ✓ done, ● running, – pending), quick controls.
- Bulk actions: pause all, resume all, retry failed. Side summary: duration, cost, errors for this Auftrag.

5.4 Position / Workflow detail

- Header: post, language pair, Plunet Item number, Workflow status, control bar (Pause, Resume, Cancel, Retry, Restart from…).
- **Overview (Observe):** vertical Task timeline with state, assignee, start/end, duration, attempts, linked Plunet Job.
- **Logs (Monitor):** live stream, filterable by Task and level (info/warn/error/action). Control actions are logged with who, when and why.
- **Analytics (Assess):** words, segments, time per Task, cost, QA warnings, errors.
- **File:** segments viewer and raw/processed XLIFF (today's content, moved here).

5.5 Analytics

- Cross-order KPIs: throughput, average duration, cost, error rate.
- Breakdowns per Project Manager and per customer; time range filter.

6. State model & controls

Task states and their mapping to Plunet `JobStatus`:

TASK STATE	MEANING	PLUNET JOBSTATUS
PENDING	Not started yet	IN_PREPERATION
RUNNING	Being executed (automatic) or worked on	IN_PROGRESS
WAITING	Waiting on a person or resource in Plunet	ASSIGNED_WAITING, REQUESTED
DONE	Finished successfully	DELIVERED, APPROVED
FAILED	Error; retry available	–
SKIPPED	Removed by override or restart	–
CANCELLED	Stopped for good	CANCELED

OVERDUE (Plunet) is shown as a flag on top of the state, not as a separate state.

Workflow states (derived from Tasks plus user actions): QUEUED · RUNNING · WAITING · PAUSED · FAILED · CANCELLED · COMPLETED. The Auftrag status is the combination of its Positions' Workflow states.

Controls

ACTION	AVAILABLE WHEN	BEHAVIOUR
Pause	RUNNING, WAITING	The running Task finishes; no new Task starts. Human Tasks keep going in Plunet.
Resume	PAUSED	Continues with the next pending Task.
Cancel	Any non-final state	Final. Cancels all unfinished Tasks and sets their Plunet Jobs to CANCELED. A reason is required.
Retry	FAILED	Reruns only the failed Task. The default action after a failure.
Restart from…	FAILED, COMPLETED, PAUSED	Reruns the Workflow from a chosen Task. Earlier results are kept; later Tasks are reset.

Every action asks for confirmation and is written to the log (actor, timestamp, reason). The actor field is ready for real users when login arrives.

7. Polylang mock server

A new Docker Compose service `polylang-mock` (FastAPI). It mirrors the documented Polylang Pro REST API and adds one clearly marked mock-only route for ready-made XLIFF files. Seed data: a few posts in several languages, with matching .xliff fixtures.

ROUTE	SOURCE	PURPOSE
GET /wp-json/pll/v1/languages	Polylang	Languages: name, slug, locale, flag_url, home_url, page_on_front, page_for_posts
GET /wp-json/wp/v2/posts?lang=xx	Polylang	Posts filtered by language; each has `lang` and `translations` {code: id}
GET /wp-json/wp/v2/posts/{id}	WP REST	Single post
POST /wp-json/wp/v2/posts/{id}?lang=fr&translations[en]=ID	Polylang	Return trip: set the language and link the translation
POST /wp-json/wp/v2/posts	WP REST	Return trip: create the translated post
GET /wp-json/pll/v1/posts/{id}/xliff?target=fr	**Mock only**	Ready-made .xliff for one post × target language

The XLIFF route is not part of the real Polylang REST API. It is isolated behind a `SourceConnector` interface so it can be replaced by the real export mechanism later.

8. Backend changes

Data model (replaces the flat ETLJob)

TABLE	KEY FIELDS
orders	id, kind (AUFTRAG|ANGEBOT), plunet_order_id / plunet_quote_id, number, customer_id, pm_resource_id, status, deadline, template_id, default_destination
positions	id, order_id, plunet_item_id, source_site, polylang_post_id, source_lang, target_lang, xliff_raw, xliff_processed, words, segments, destination_override
workflows	id, position_id, template_id, status, started_at, finished_at
tasks	id, workflow_id, seq, type (IMPORT|PM|TRA|REV|QA|DELIVER|...), status, plunet_job_id, assignee_resource_id, attempts, cost, started_at, finished_at, error
workflow_events	id, workflow_id, task_id, level, kind (log|state|action), message, actor, reason, created_at
workflow_templates	id, name, task chain (ordered task types)
xliff_segments	Kept, re-linked to positions

Execution

- Replace the synchronous `/etl/run` with an **async worker** that reads Workflow/Task state from Postgres, so pause, resume, retry and restart are state transitions.
- Human Tasks (PM, TRA, REV) are synced with Plunet Jobs through polling and/or Plunet event callbacks.
- Live updates to the UI via Server-Sent Events (logs and state).
- Plunet resources (PMs) and customers are read through the existing SOAP client.

API (sketch)

```
GET  /api/v1/orders                    ?kind&status&pm&customer&attention
POST /api/v1/orders                    create Auftrag/Angebot (+ Positions)
GET  /api/v1/orders/{id}
POST /api/v1/orders/{id}/accept        Angebot -> Auftrag
POST /api/v1/orders/{id}/actions       bulk pause/resume/retry
GET  /api/v1/positions/{id}
POST /api/v1/workflows/{id}/actions    {action, from_task?, reason?}
GET  /api/v1/workflows/{id}/events     paginated; /stream for SSE
GET  /api/v1/analytics                 ?group_by=pm|customer&from&to
GET  /api/v1/plunet/resources | /customers
GET  /api/v1/sources/polylang/...      proxy to the Polylang connector
GET  /api/v1/templates
```

9. Frontend changes

- Split `App.tsx` into routed pages (React Router) and shared components: StatusTag, TaskStepper, ControlBar, LogStream, KpiTile.
- **i18n:** English and French with a header switch, using react-i18next. All strings live in locale files; dates, numbers and currency are formatted per locale.
- Keep the existing theme tokens from DESIGN.md unchanged.
- Prepare for auth: no hard-coded user; the actor comes from one place, and “My Aufträge” can later filter by the logged-in PM.

10. Implementation phases

PHASE	CONTENT	OUTPUT
0	Design artifact: Aufträge list, wizard, Auftrag detail, Position/Workflow detail	Clickable hi-fi mock for approval
1	Polylang mock service + seed data + SourceConnector	Docker service, endpoints in section 7
2	New data model + migrations; Plunet order/quote/item/job creation	orders / positions / workflows / tasks
3	Async workflow engine, controls, event log, SSE	Pause/resume/cancel/retry/restart working
4	Frontend rebuild: routes, pages, components, i18n EN/FR	New UX live
5	Angebot acceptance (app + Plunet), return trip options	Quote → order; Polylang push-back
6	Analytics (per order, PM, customer)	Analytics page
Later	Login and role-based views	Auth, “My Aufträge”

11. Points to confirm during design review

– Labels in the EN/FR UI: proposed **Order / Quote** (EN) and **Commande / Devis** (FR), with the Plunet terms Auftrag/Angebot shown as a hint.
– Exact list of extra Plunet task types to offer in templates beyond PM, TRA, REV, QA.
– Cost source: Plunet prices per Job, or rates configured in FOX MLO.